VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS ENGINEERING

------o0o-----

CAPSTONE PROJECT 2

TOPIC

DESIGN AND IMPLEMENTATION OF A PIPELINED RISC-V PROCESSOR INTEGRATING TILELINK-UL BUS PROTOCOL

Advisor: Assoc. Prof. Dr. Trần Hoàng Linh

Student: Nguyễn Duy Ngọc

Student ID: 2251036

Ho Chi Minh City, December 2025

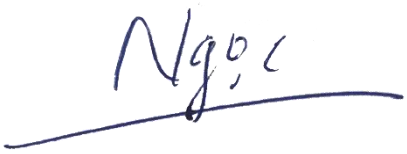
ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my advisor, **Assoc. Prof. Dr. Trần Hoàng Linh**, for his invaluable guidance, continuous encouragement, and insightful technical advice throughout the duration of this project. His expertise and dedicated mentorship have been instrumental in the successful completion of this research.

I would also like to extend my sincere thanks to my classmates and colleagues for their companionship and collaboration. The technical discussions and mutual support we shared were vital in overcoming the challenges encountered during the design and implementation phases.

Finally, I am profoundly grateful to my family for their unwavering support and belief in me, which provided the necessary motivation to achieve my academic goals.

Ho Chi Minh City, December 31st, 2025



Nguyễn Duy Ngọc

PROJECT SUMMARY

This project presents the architectural enhancement of a 32-bit RISC-V processor through the integration of the TileLink Uncached Lightweight (TL-UL) bus protocol. In the preceding design phase, the processor's Load-Store Unit (LSU) interfaced with the Data Memory via a direct, tightly coupled connection, which restricted system modularity and hindered the integration of peripheral devices. To address these limitations, this study implements a standardized interconnect framework comprising a Host Adapter, a Device Adapter, and specialized interface logic. A significant technical contribution of this work is the development of an address translation mechanism capable of managing misaligned memory accesses within an interleaved four-bank memory architecture while interfacing with a flat-addressed bus. The functional correctness of the design was verified through rigorous simulation. Furthermore, the system was synthesized and implemented on an FPGA platform, specifically utilizing the Intel Cyclone V 5CSXFC6D6F31C6 device on the DE10-Standard development kit, using Intel Quartus Prime software to validate hardware feasibility and resource utilization.

TABLE OF CONTENTS

TABLE OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1. Overview

The RISC-V instruction set architecture (ISA) has rapidly become a standard in modern digital design due to its open-source nature and modular extensibility. In a previous capstone project, a 32-bit RISC-V processor utilizing a classic 5-stage pipeline (Fetch, Decode, Execute, Memory, and Writeback) was successfully implemented and verified. While that design demonstrated the functional correctness of the core processing unit, the interface between the Load-Store Unit (LSU) and the Data Memory relied on a custom, ad-hoc direct connection. This architectural choice, although simple, created a rigid dependency between the processor core and the specific memory implementation, thereby hindering future expansion or the integration of additional peripheral devices.

To address these scalability challenges, this project focuses on decoupling the processor core from the memory subsystem by adopting an industry-standard bus protocol. TileLink, specifically the Uncached Lightweight (TL-UL) profile from the OpenTitan project, was selected for its suitability for low-complexity, low-latency microcontroller applications. The integration of TL-UL transforms the proprietary interface into a standardized transaction-based communication channel, facilitating a more robust and modular System-on-Chip (SoC) design.

1.2. Project Objectives

The primary objective of this project is to design and implement the necessary hardware components to enable TileLink communication within the existing RISC-V processor. To achieve the stated objectives, the project workload is divided into four primary technical tasks.

Task 1: Protocol Analysis and Specification

The initial phase involves a comprehensive analysis of the TileLink-UL specification to define the signal mapping requirements for the processor's Load-Store Unit. This task necessitates a detailed study of the handshake mechanisms on Channel A (Request) and Channel D (Response) to ensure strict adherence to timing and flow control standards. The outcome of this phase is a defined architectural specification for the adapters that bridges the processor's control logic with the bus protocol.

Task 2: Adapter Design and Implementation

The second task focuses on the Register-Transfer Level (RTL) implementation of the connectivity modules using SystemVerilog. A Host Adapter is designed to serialize the processor's parallel memory requests into sequential TileLink packets, while a Device Adapter is implemented to decode these packets back into memory control signals. Particular attention is given to the generation of accurate byte-enable

masks to support byte, half-word, and word operations as required by the RISC-V ISA.

Task 3: Interleaved Memory Interface Logic Design

The third task addresses the critical challenge of handling misaligned memory accesses. Since the TileLink bus employs a flat addressing scheme, whereas the physical data memory is organized into four interleaved 8-bit banks, specialized interface logic is required. This task involves designing an address recalculation circuit positioned after the Device Adapter. This logic dynamically generates independent address indices for each memory bank, enabling the correct execution of misaligned store and load instructions without stalling the processor pipeline.

Task 4: Verification and FPGA Implementation

The final task integrates the designed components into the top-level processor hierarchy and validates the system through a two-stage verification process. First, functional verification is performed using simulation tools to analyze waveform timing and data integrity. Subsequently, the design is synthesized and implemented on the Intel Cyclone V 5CSXFC6D6F31C6 FPGA (DE10-Standard kit) using Intel Quartus Prime. This step confirms that the integrated system meets timing constraints and fits within the logic resource budget of a physical hardware device.

2. THEORETICAL BACKGROUND

2.1. RISC-V Processor Architecture

The foundation of this project is the 32-bit RISC-V processor designed in the preceding capstone phase. This processor implements the RV32I base integer instruction set architecture, which defines a load-store architecture with 32 general-purpose registers. The core microarchitecture follows a classic five-stage pipeline organization, comprising Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Writeback (WB).

In the context of this study, the Memory Access (MEM) stage is of particular importance. In the original design, the Load-Store Unit (LSU) within this stage interfaced directly with the Data Memory using a proprietary set of control signals. While efficient for simple systems, this direct coupling lacks the standardization required for complex System-on-Chip (SoC) integration. Therefore, understanding the timing and signal requirements of the LSU is crucial for designing the subsequent bus adapters. The high-level datapath of the processor is illustrated in *Figure 1*.

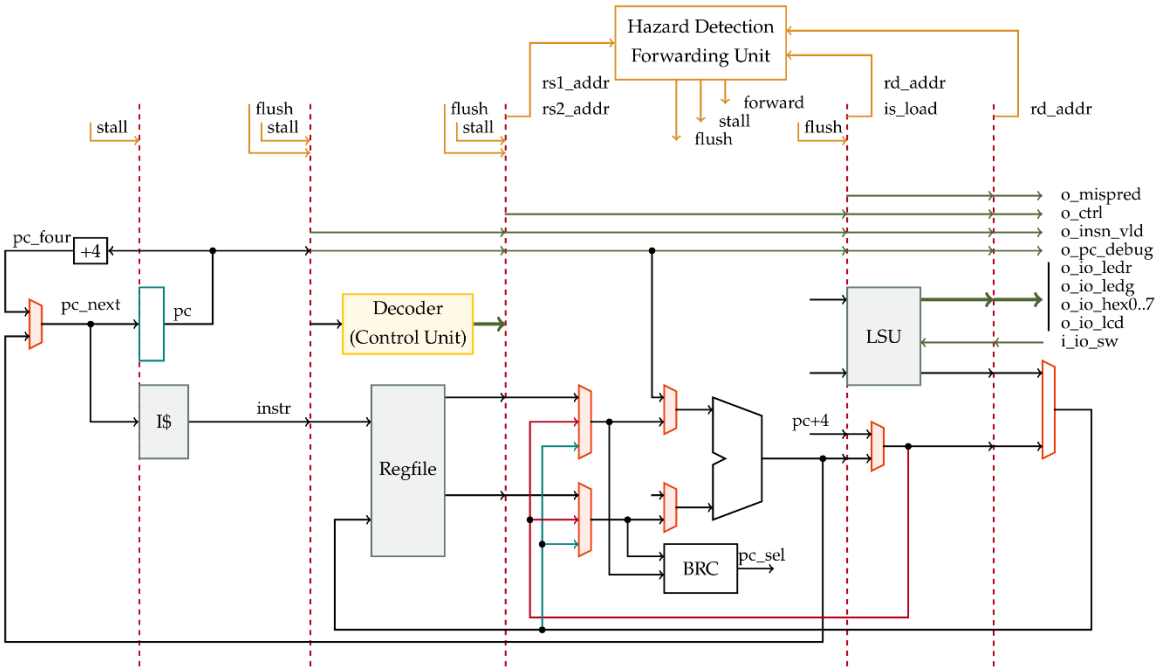


Figure 1. Datapath of 5-Stage RV32I Pipelined Processor with Full Hazard Detection, Forwarding, and Control Logic

2.2. TileLink Uncached Lightweight (TL-UL) Protocol

2.2.1. Protocol Overview

TileLink is a free and open chip-scale interconnect standard designed for low-latency and high-throughput communication. This project specifically utilizes the TileLink Uncached Lightweight (TL-UL) profile, which is optimized for simple memory-mapped devices and microcontroller-class processors. Unlike complex

cache-coherent protocols, TL-UL focuses on simplicity and efficiency, supporting only two types of transactions: Get (Read) and Put (Write). The protocol operates on a master-slave topology where the processor acts as the host and the memory acts as the device.

2.2.2. Channel Architecture

The communication in TL-UL is bidirectional but decoupled into two independent channels: Channel A and Channel D. Channel A flows from the master to the slave and carries request messages, such as address and write data. Conversely, Channel D flows from the slave to the master and carries response messages, including read data and operation acknowledgments. This separation allows for flexible flow control and potential pipelining of requests. The key signals defining these channels are summarized in *Table 1*.

Table 1. Key Signals of the TileLink-UL Protocol

Channel	Signal Name	Direction	Description
Channel A	a_valid	Master to Slave	Indicates that the request information is valid.
	a_ready	Slave to Master	Indicates that the slave is ready to accept the request.
	a_opcode [2:0]	Master to Slave	Specifies the operation type (Get, PutFullData, PutPartialData).
	a_addr [AW-1:0]	Master to Slave	The target byte address for the transaction.
	a_data [DW-1:0]	Master to Slave	The data payload to be written (for Put operations).
	a_mask [DBW-1:0]	Master to Slave	Byte enable mask for writing specific bytes within a word.
Channel D	d_valid	Slave to Master	Indicates that the response information is valid.
	d_ready	Master to Slave	Indicates that the master is ready to accept the response.
	d_opcode [2:0]	Slave to Master	Specifies the response type (AccessAck, AccessAckData).
	d_data [DW-1:0]	Slave to Master	The data payload read from memory (for Get operations).

2.2.3. Handshake Mechanism

The Valid-Ready Protocol: Reliable data transfer in TileLink is governed by a strict decoupled handshake mechanism that applies independently to every channel. The protocol relies on two fundamental signals:

valid: Asserted by the source to indicate that the payload on the bus is stable and valid.

ready: Asserted by the sink to indicate that it is capable of accepting the payload in the current clock cycle.

A successful transaction, known as a "beat," occurs only on the rising edge of the clock where both valid and ready signals are logically high. This mechanism decouples the timing of the master and slave, allowing components with different operating frequencies or latencies to communicate without data loss.

Channel A Handshake (Request Flow): *Figure 2* illustrates the handshake sequence on Channel A, where the Master sends request messages to the Slave.

Scenario 1 (Master Wait): The Master asserts a_valid first, but the Slave is busy (a_ready is low). The Master must hold the address and control signals stable until the Slave asserts a_ready.

Scenario 2 (Slave Wait): The Slave asserts a_ready (indicating it is idle), but the Master has no request (a_valid is low). No transaction occurs.

Scenario 3 (Transfer): When both a_valid and a_ready are high, the request is transferred.

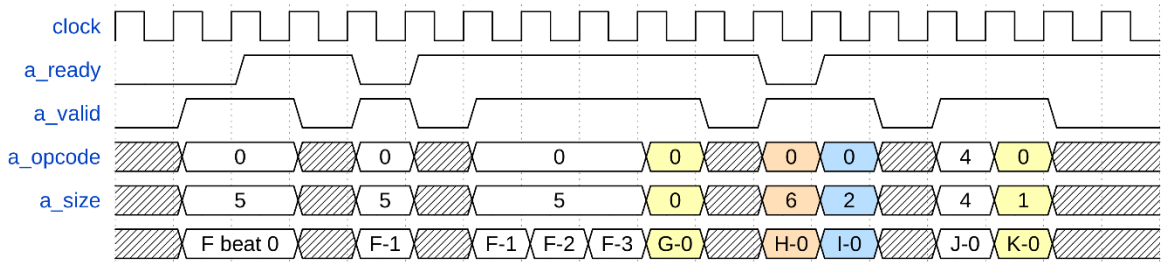


Figure 2. Ready-Valid signalling for 6 messages in a 64-bit Channel A[4]

Channel D Handshake (Response Flow): Conversely, *Figure 3* depicts the handshake on Channel D, where the Slave returns data or acknowledgments to the Master. The roles are reversed: the Slave drives d_valid and the Master drives d_ready. Crucially, the protocol forbids a dependency loop where the valid signal

waits for the ready signal to be asserted. However, the ready signal is permitted to wait for valid. This rule prevents deadlock situations in the system.

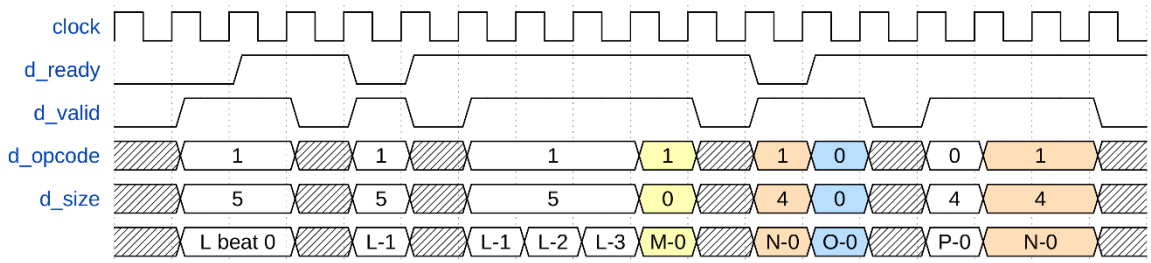


Figure 3. Ready-Valid signalling for 6 messages in a 64-bit Channel D[4]

2.3. Interleaved Memory Organization

To support the RISC-V specification for misaligned memory accesses (e.g., storing a word across two word boundaries), the physical data memory is not designed as a single 32-bit block. Instead, it is organized as four independent 8-bit memory banks. This interleaved architecture allows the system to access bytes at different row indices simultaneously. However, the TileLink bus transmits a single unified address. Consequently, a theoretical framework for "address swizzling" is required to translate the linear bus address into four distinct bank addresses. This theory forms the basis for the glue logic design presented in the subsequent chapter.

3. DESIGN AND IMPLEMENTATION

3.1. Design Specifications

To ensure the successful integration of the bus protocol into the RISC-V processor, the hardware design must satisfy specific quantitative and qualitative technical requirements. These specifications are derived from the constraints of the target FPGA platform and the operational characteristics of the 5-stage pipeline.

Bus Protocol Specification: The system must implement the TileLink Uncached Lightweight (TL-UL) profile. It must support two primary channel types: Channel A for requests and Channel D for responses, operating with a valid-ready handshake mechanism.

Memory Interface Specifications: The system must interface with a 64 KB Data Memory (2^{16} bytes), mapped from address 0x0000_0000 to 0x0000_FFFF. The physical memory must be organized as four 8-bit banks to support byte-level granularity.

Latency Constraints: The introduction of the bus adapters must not induce pipeline stalls during standard aligned memory accesses. The total latency for a load/store operation through the bus infrastructure should be deterministic.

Misalignment Handling: The design must support misaligned word and half-word accesses (e.g., a Store Word instruction at address 0x0002). The logic must automatically split and route data to the appropriate memory banks within a single clock cycle to avoid multi-cycle structural hazards.

Hardware Platform: The synthesized design must fit within the logic resources of the Intel Cyclone V 5CSXFC6D6F31C6 FPGA (DE10-Standard Kit) and operate stably at a clock frequency of 50 MHz.

3.2. Design Analysis and Methodology Selection

To determine the optimal interconnect architecture, two distinct methods were analyzed based on scalability, complexity, and performance.

Method 1: Direct Coupled Interface (Legacy Approach) This method involves connecting the processor's Load-Store Unit directly to the memory ports.

Advantages: This approach offers the lowest possible latency and minimal hardware resource usage, as no intermediate logic is required. It is simple to implement for a single-core, single-memory system.

Disadvantages: It lacks modularity. If the memory implementation changes (e.g., from on-chip SRAM to external DDR), the processor core logic must be modified. Furthermore, adding peripherals requires complex custom multiplexing logic, leading to routing congestion.

Method 2: TileLink-UL Bus-Based Interface (Proposed Approach) This method introduces a standardized bus protocol between the master and the slave.

Advantages: This approach completely decouples the processor from the memory implementation. It allows for easy expansion; new peripherals can be added by simply attaching them to the bus crossbar without modifying the CPU core. It adheres to open standards, facilitating the use of open-source IP blocks.

Disadvantages: It introduces additional latency due to the handshake logic and consumes more logic resources (LUTs) for the adapter implementation.

Conclusion: While Method 1 is sufficient for simple educational cores, Method 2 is selected for this project. The slight increase in resource usage is justified by the significant gains in modularity and architectural compliance, which aligns with the project's goal of developing a scalable SoC foundation.

3.3. Top-Level System Architecture

3.3.1. General Description

The proposed system architecture integrates the legacy RISC-V core with the TileLink communication infrastructure. Unlike the previous direct-mapped architecture, this design introduces an abstraction layer—the Interconnect Manager—between the processor's execution pipeline and the physical memory banks. This modular approach ensures that the processor operates as a standard TileLink Master, while the memory subsystem functions as a compliant Slave.

3.3.2. Top-Level Block Diagram

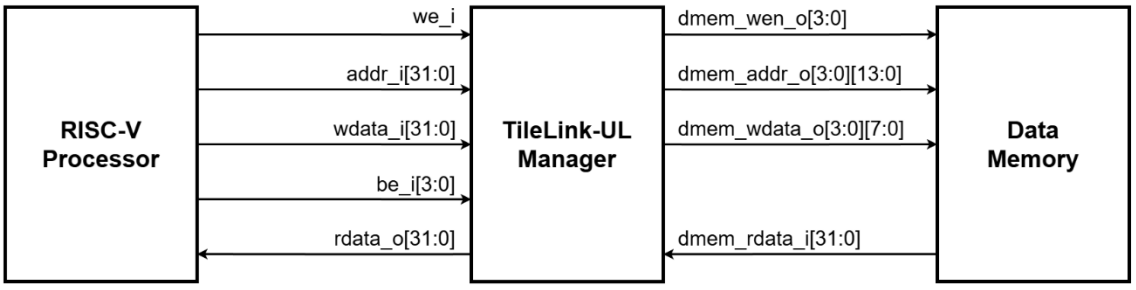


Figure 4. Top-Level System Architecture

3.3.3. System Interface Specifications

The top-level signal interface remains compatible with the previous FPGA implementation requirements, ensuring seamless porting to the DE10-Standard development kit.

Table 2. Top-Level Interface Signals

Signal Group	Signal Name	Width	Direction	Description
System	clk	1-bit	Input	50 MHz System Clock source.
	rstn	1-bit	Input	Active-low Asynchronous Reset.
Input Peripherals	SwDataIn	32-bit	Input	Data input from board's slide switches.
	KeyDataIn	2-bit	Input	Data input from push buttons (for interrupts/control).
Output Peripherals	LedrDataOut	32-bit	Output	Control signals for Red LEDs (Memory Mapped).
	LedrDataOut	32-bit	Output	Control signals for Green LEDs (Memory Mapped).
	LcdDataOut	32-bit	Output	Data interface for the LCD display module.
	HexDataOut[0..7]	8 x 7-bit	Output	Control signals for the 8 seven-segment display blocks.
Debug Trace	pc	32-bit	Output	Real-time Program Counter value for debugging/waveform.
	InstrVld	1-bit	Output	Instruction Valid flag (indicates a completed instruction).
	MisPred	1-bit	Output	Branch Misprediction flag (monitors branch predictor accuracy).
	Ctrl	1-bit	Output	Control Instruction flag (indicates Branch/Jump execution).
	Ecall	1-bit	Output	Environment Call trap signal (System Call).
	Ebreak	1-bit	Output	Environment Break trap signal (Debugger Breakpoint).

3.4. TileLink Interconnect Manager (tlul_mgr)

3.4.1. Block Diagram

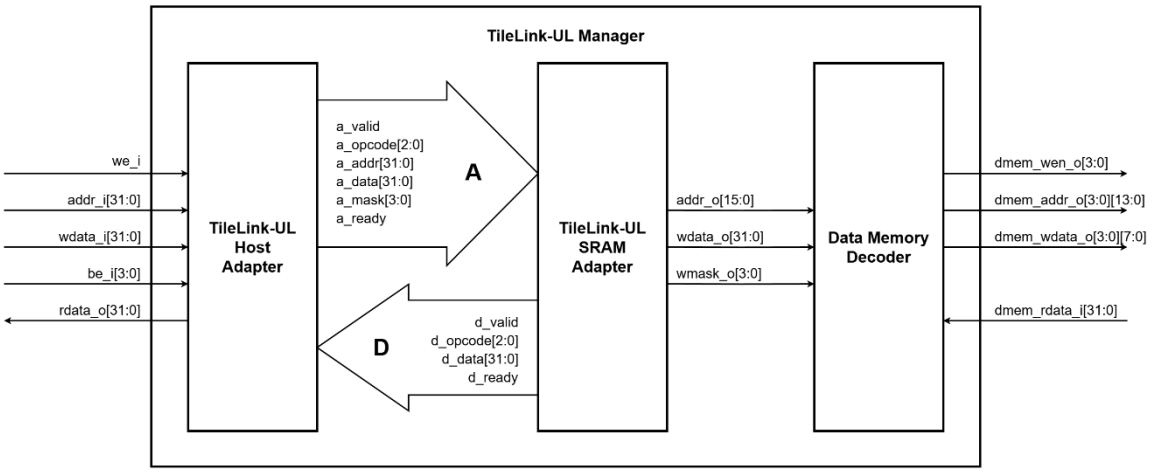


Figure 5. Internal Structure of TileLink Manager

3.4.2. Principle of Operation

The tlul_mgr acts as a physical abstraction layer. While it primarily routes signals, it enforces the standard interface between the processor and the memory subsystem.

Signal Mapping: The module maps the processor's 32-bit addr_i directly to the Host Adapter. Crucially, the write mask be_i (4-bit) allows the system to support byte-granularity operations (SB, SH, SW) without internal Read-Modify-Write cycles.

Response Routing: The read data dmem_rdata_i from the physical memory is fed into the Device Adapter, which then packets it into the TileLink Channel D format (d_data) to be returned to the processor via rdata_o.

3.4.3. Interface Specifications

Table 3. TileLink Manager Signals (tlul_mgr)

Interface	Signal Name	Width	Direction	Description
System	clk	1-bit	Input	System clock signal.
	rstn	1-bit	Input	Active-low asynchronous reset.
Processor Side	req_i	1-bit	Input	Memory access request strobe (from LSU).
	we_i	1-bit	Input	Write Enable control (1 = Write, 0 = Read).
	addr_i	32-bit	Input	Target byte address (from ALU Result).

	wdata_i	32-bit	Input	Data to be written to memory.
	be_i	4-bit	Input	Byte-enable mask (Active high).
	rdata_o	32-bit	Output	Data read from memory returned to CPU.
Physical Memory Side	dmem_addr_o	4 x 14-bit	Output	Independent physical addresses for 4 memory banks (Decoded).
	dmem_wdata_o	4 x 8-bit	Output	Write data split for 4 memory banks.
	dmem_wen_o	4-bit	Output	Independent Write Enable for each bank.
	dmem_rdata_i	32-bit	Output	Combined read data from 4 memory banks.

3.5. Host Adapter Design (tlul_adapter_host)

3.5.1. Block Diagram

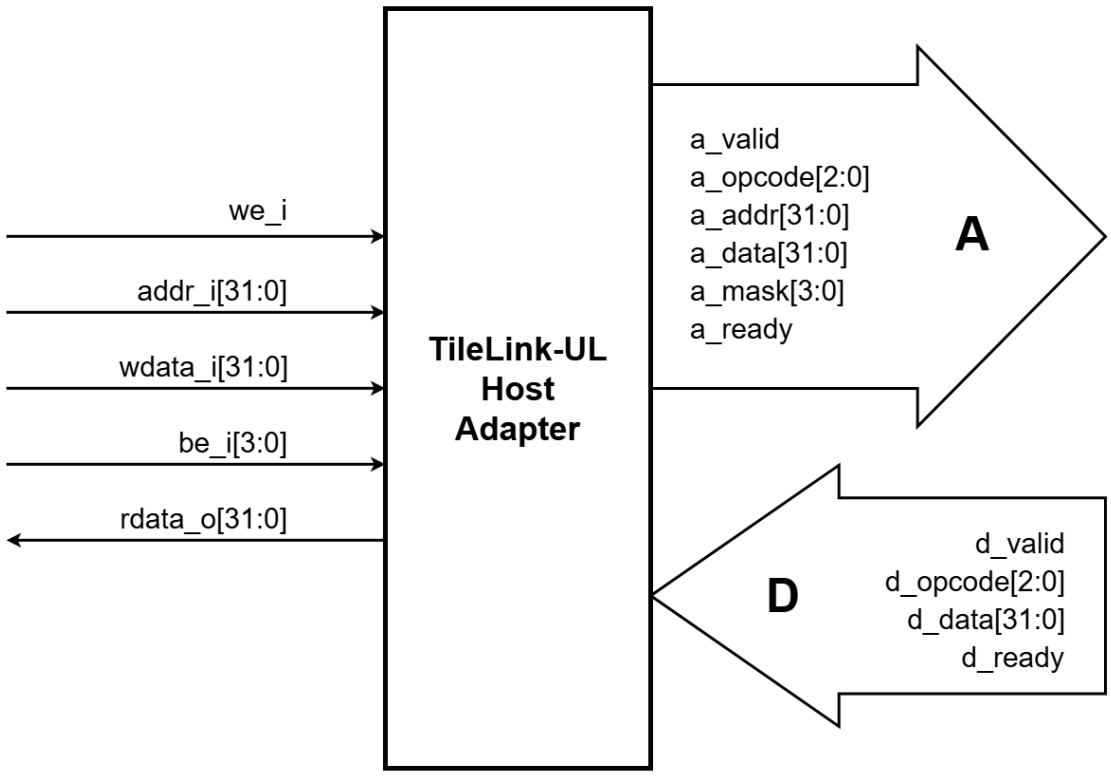


Figure 6. Host Adapter Logic Flow

3.5.2. Principle of Operation

The Host Adapter logic is responsible for state translation between the processor's static control signals and the bus's dynamic handshake protocol. The internal logic operates as follows:

Opcode Generation Logic: The adapter determines the type of bus transaction based on the we_i (Write Enable) and be_i (Byte Enable) inputs:

Get (Read): If we_i is logic low (0), the a_opcode is set to Get. The a_mask is forced to all ones to request a full word read.

PutFullData (Write Word): If we_i is high (1) and be_i is 4'b1111, the a_opcode is set to PutFullData.

PutPartialData (Write Byte/Half): If we_i is high (1) and be_i is not 1111 (e.g., 0001 or 0011), the a_opcode is set to PutPartialData. This distinction is critical for the slave to know whether to overwrite the entire word or specific bytes.

Handshake and Flow Control:

Request Phase: When the Load-Store Unit asserts req_i, the adapter asserts a_valid on Channel A. It holds this signal high until the bus responds with a_ready.

Response Phase: The adapter monitors Channel D. Upon detecting d_valid high, it captures the d_data payload and routes it to rdata_o. The valid_o signal is then asserted to inform the processor pipeline that the memory operation is complete.

3.5.3. Interface Specifications

Table 4. Host Adapter Interface (tlul_adapter_host)

Interface	Signal Name	Width	Direction	Description
System	clk	1-bit	Input	System clock signal.
	rstn	1-bit	Input	Active-low asynchronous reset.
Processor Side	req_i	1-bit	Input	Memory access request strobe (from LSU).
	we_i	1-bit	Input	Write Enable control (1 = Write, 0 = Read).
	addr_i	32-bit	Input	Target byte address (from ALU Result).
	wdata_i	32-bit	Input	Data to be written to memory.
	be_i	4-bit	Input	Byte-enable mask (Active high).

	rdata_o	32-bit	Output	Data read from memory returned to CPU.
	valid_o	1-bit	Output	Transaction completion flag (optional).
TL-UL Channel A (Request)	tl_o.a_valid	1-bit	Output	Master Valid: Request is valid.
	tl_o.a_opcode	3-bit	Output	Operation type (Get, PutFull, PutPartial).
	tl_o.a_address	32-bit	Output	Target address on the bus.
	tl_o.a_mask	4-bit	Output	Byte lane mask for the bus.
	tl_o.a_data	3-bit	Output	Write data payload on the bus.
	tl_i.a_ready	1-bit	Input	Slave Ready: Slave accepts the request.
TL-UL Channel D (Response)	tl_i.d_valid	1-bit	Input	Slave Valid: Response is available.
	tl_i.d_opcode	3-bit	Input	Response type (AccessAck, AccessAckData).
	tl_i.d_data	32-bit	Input	Read data payload from the bus.
	tl_o.d_ready	1-bit	Output	Master Ready: Host accepts response.

3.6. Device Adapter Design (tlul_adapter_sram)

3.6.1. Block Diagram

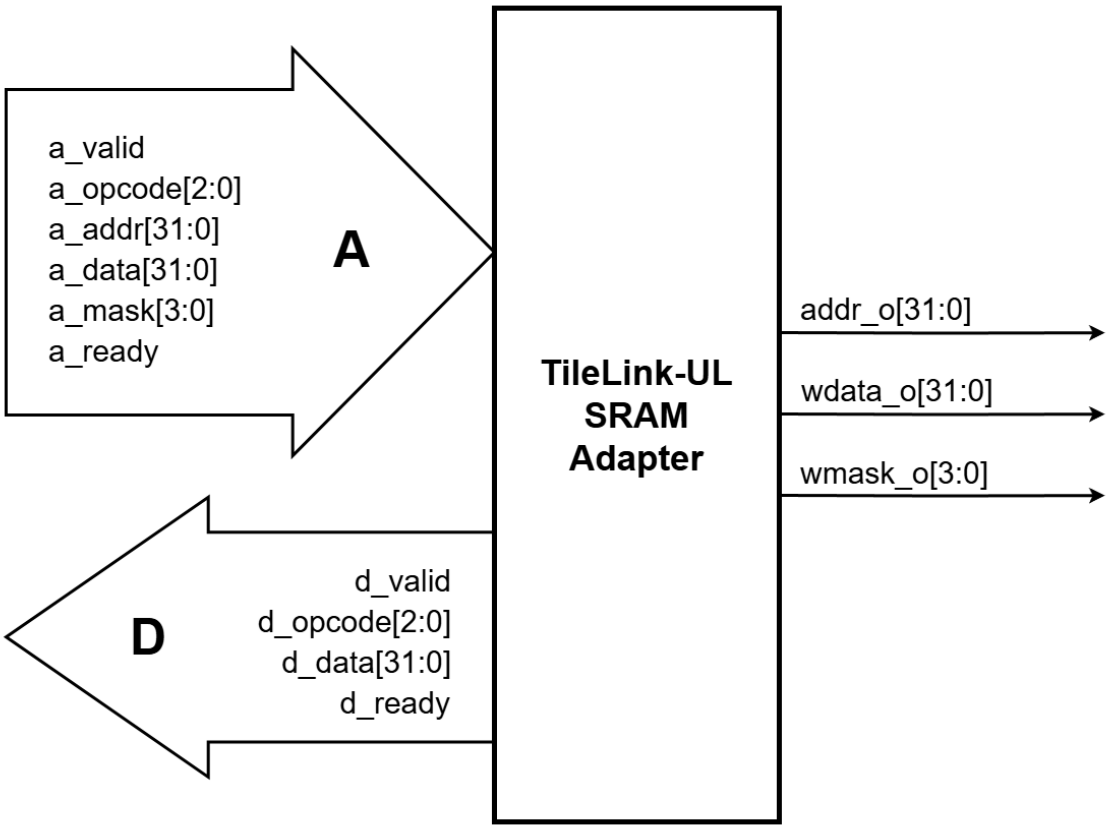


Figure 7. Device Adapter Logic Flow

3.6.2. Principle of Operation

The Device Adapter functions as a decoder and state manager for the memory. Its operation is divided into address processing and command decoding.

Address Truncation Logic: The generic TileLink bus carries a 32-bit address (a_addr). However, the tlul_adapter_sram is parameterized with SramAw = 16. The logic automatically truncates the upper 16 bits of the incoming address, outputting only addr_o[15:0]. This ensures that the generated address falls strictly within the 64 KB physical address space of the target memory.

Command Decoding: The adapter decodes the Channel A packet to drive the SRAM control lines:

Write Operation: When a_opcode indicates a Put transaction, the adapter asserts we_o (Write Enable) and passes the a_mask to wmask_o.

Read Operation: When a_opcode indicates a Get transaction, we_o is de-asserted, configuring the SRAM for a read cycle.

Synchronous Acknowledgment: Since the target Block RAM is synchronous with a fixed 1-cycle latency, the adapter implements a simplified acknowledgment loop. The req_o signal (SRAM Chip Select) is looped back to drive rvalid_i. This informs the bus manager that the read data (rdata_i) or write acknowledgment is valid in the immediate next clock cycle, maximizing throughput.

3.6.3. Interface Specifications

Table 5. Device Adapter Interface (tlul_adapter_sram)

Interface	Signal Name	Width	Direction	Description
System	clk	1-bit	Input	System clock signal.
	rstn	1-bit	Input	Active-low asynchronous reset.
TL-UL Channel A (Request)	tl_i.a_valid	1-bit	Input	Master Valid: Request available from bus.
	tl_i.a_opcode	3-bit	Input	Operation type (PutFull, Get, etc.).
	tl_i.a_address	32-bit	Input	Full bus address.
	tl_i.a_mask	4-bit	Input	Byte lane mask from bus.
	tl_i.a_data	3-bit	Input	Write data payload from bus.
	tl_o.a_ready	1-bit	Output	Slave Ready: Adapter is ready to process.
TL-UL Channel D (Response)	tl_o.d_valid	1-bit	Output	Slave Valid: Response is available.
	tl_o.d_opcode	3-bit	Output	Response type (AccessAck, AccessAckData).
	tl_o.d_data	32-bit	Output	Read data payload returning to Master.
	tl_i.d_ready	1-bit	Input	Master Ready: Bus accepts the response.
SRAM Interface (To Glue Logic)	req_o	1-bit	Output	Chip Select: Valid memory request strobe.
	we_o	1-bit	Output	Write Enable (1=Write, 0=Read).
	addr_o	16-bit	Output	Truncated physical address (2^{16} space).
	wdata_o	32-bit	Output	Write data passed to memory logic.
	wmask_o	4-bit	Output	Byte write mask (derived from a_mask).

	rdata_i	32-bit	Input	Read data returned from memory banks.
	rvalid_i	1-bit	Input	Read valid acknowledgement (looped back from req_o).

3.7. Address Decoupling Logic (dmem_dec)

3.7.1. Block Diagram

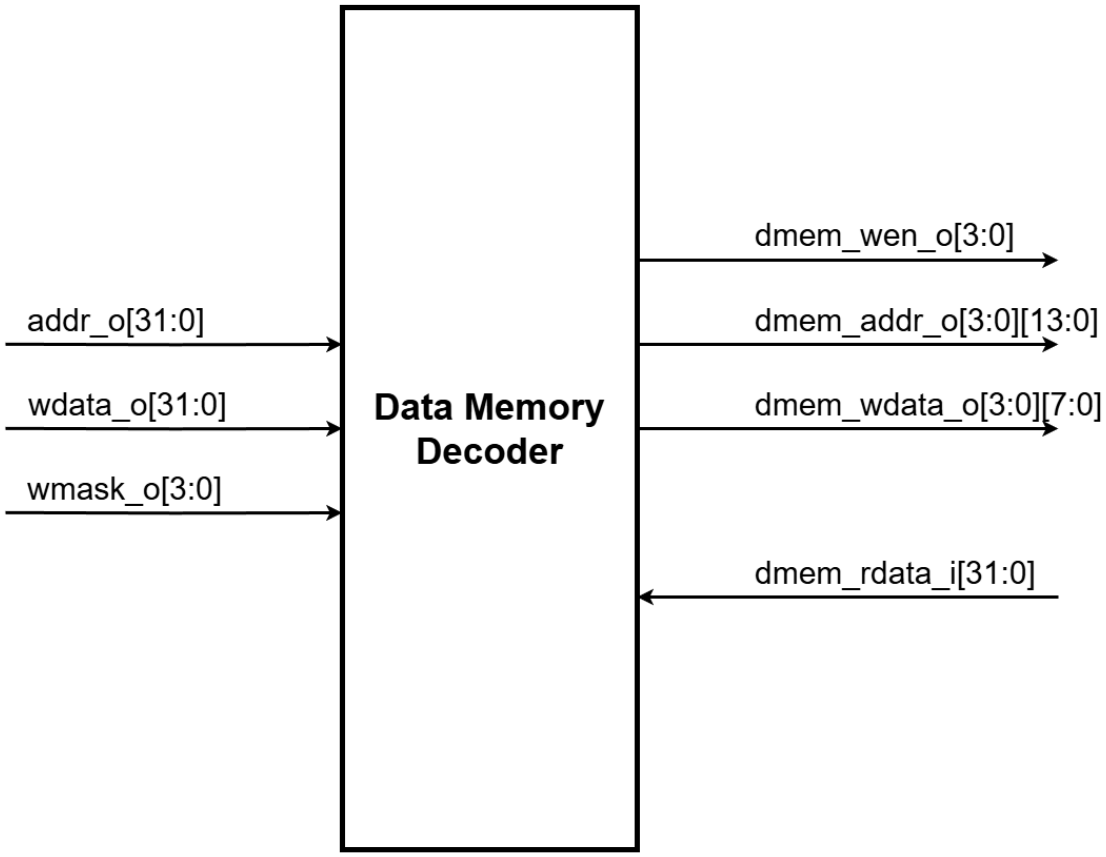


Figure 8. Address Decoupling Circuit

3.7.2. Principle of Operation

This module implements the "Address Swizzling" algorithm required to interface the linear bus with the interleaved memory banks. The logic handles three distinct memory access scenarios based on the input address sram_addr_i and write mask sram_wmask_i.

Scenario 1: Aligned Access (Offset = 0)

Input Condition: The two least significant bits (LSB) of the address are 00.

Logic: The Selector Decoder asserts sel[0]. The multiplexing logic routes the Base Index (idx_base) to all four memory banks (dmem_addr_o[0..3]).

Result: A standard Word Read/Write operation occurs at row N.

Scenario 2: Misaligned Access (e.g., Offset = 2)

Input Condition: The address LSBs are 10. This implies a Half-word or Word operation starting at byte 2.

Logic: The Selector Decoder asserts sel[2].

Banks 2 and 3 (Bytes 2 and 3 of the current word) receive the Base Index (idx_base).

Banks 0 and 1 (Bytes 0 and 1 of the *next* word) receive the Next Index (idx_next), calculated by the adder.

Result: The data is correctly striped across the end of row N and the beginning of row N+1 in a single cycle.

Scenario 3: Bank-Specific Write Enable To prevent data corruption during partial writes (e.g., Store Byte), the logic gates the global write enable with the specific byte mask.

Logic: dmem_wen_o[k] = sram_we_i AND sram_wmask_i[k] (for k=0..3).

Result: Even if the address logic points to a valid row, a memory bank will only perform a write operation if its corresponding mask bit is set. This protects adjacent bytes during fine-grained memory operations.

3.7.3. Interface Specifications

Table 6. Address Decoupling Logic Interface (dmem_dec)

Signal Name	Width	Direction	Description
sram_addr_i	16-bit	Input	Base address received from Device Adapter.
sram_wdata_i	32-bit	Input	Full 32-bit write data payload.
sram_wmask_i	4-bit	Input	Write mask (strobe) from Device Adapter.
dmem_addr_o	4 x 14-bit	Output	Independent row index calculated for each of the 4 banks.
dmem_wdata_o	4 x 8-bit	Output	Write data sliced into 8-bit segments for each bank.
dmem_wen_o	4-bit	Output	Independent write enable for each bank (gated by logic).

4. IMPLEMENTATION RESULTS

4.1. Functional Verification Results

To validate the architectural integrity of the processor and the correctness of the TileLink integration, the design underwent rigorous functional simulation. The verification environment was established using the **Cadence Xcelium Parallel Simulator**, a standard EDA tool for high-performance logic simulation. The verification methodology combined automated self-checking test suites with manual waveform inspection via **SimVision**.

4.1.1. Waveform Analysis with SimVision

Before running exhaustive regression tests, the critical handshaking protocols of the TileLink bus were visually verified using the SimVision waveform viewer. As the integration of the bus adapters transforms the processor's static control signals into dynamic transactions, verifying the timing relationship between signals is paramount.

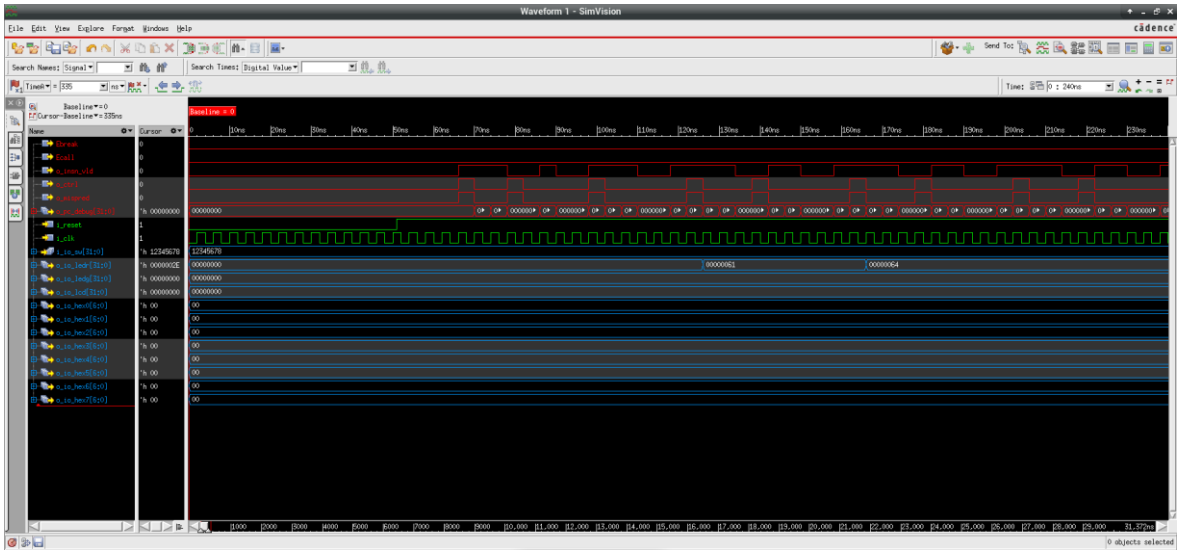


Figure 9. SimVision Waveform demonstration.

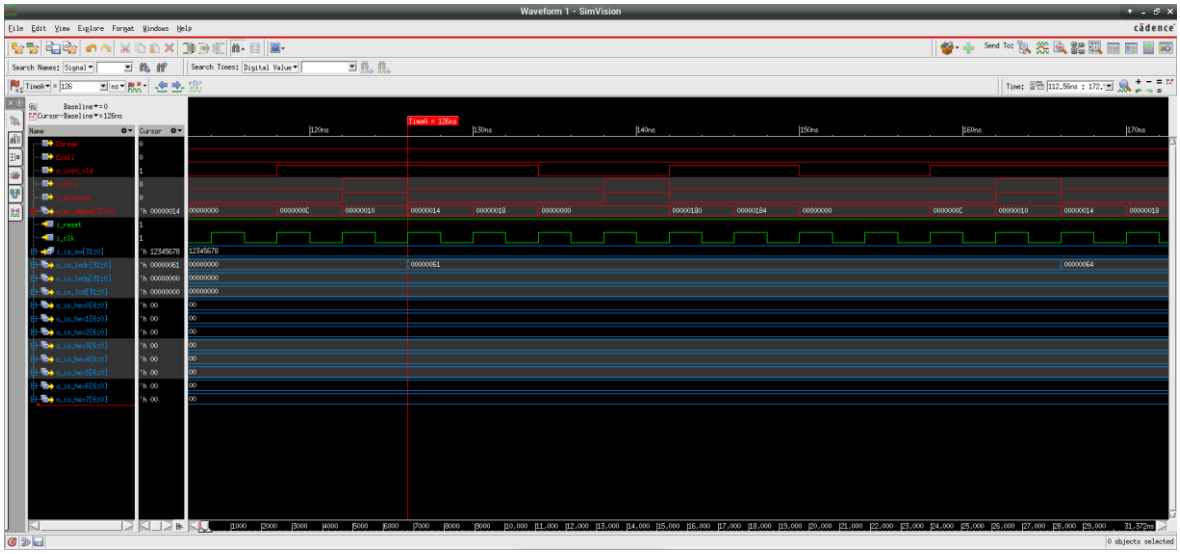


Figure 10. SimVision Waveform demonstration (cont.)

4.1.2. Verification Methodology and Test Program Logic

To ensure comprehensive fault coverage, the verification process relies on a sophisticated self-checking test suite (isa_4b.hex). An analysis of the test program's assembly code reveals that it does not merely execute random instructions; rather, it adheres to a structured "Execute-Compare-Branch" verification paradigm.

Test Logic Structure: For every instruction under test (e.g., LW - Load Word), the program executes a sequence of directed tests covering various corner cases (such as positive values, negative values, and zero). The verification logic follows a specific control flow:

Execution: The processor executes the target instruction, storing the result in a temporary register.

Validation: The program compares this result against a pre-calculated expected value (Oracle) using branch instructions like bne (Branch if Not Equal).

Branching:

Failure Path: If a mismatch occurs, the program branches to an error handling routine. It loads the ASCII sequence for "FAIL" and halts execution.

Success Path: If the result matches, the program proceeds to the next test case.

Completion: Upon passing all test vectors for a specific instruction, the program jumps to a logging routine.

Memory-Mapped I/O Logging Mechanism A critical aspect of this verification environment is the mechanism used to report results from the simulated hardware to the user terminal. Since the processor cannot directly access the simulation console, it utilizes Memory-Mapped I/O (MMIO).

Hardware Side: When the software determines a "PASS" or "FAIL" condition, it writes the corresponding ASCII characters (e.g., 'P', 'A', 'S', 'S') byte-by-byte to a specific memory address mapped to the Output Peripheral (specifically the Red LED output port at 0x10000000 in this simulation map).

Testbench Side: The simulation environment (scoreboard.sv) monitors this specific I/O port. Whenever the o_insn_vld signal is asserted and a write operation is detected at the debug address, the testbench intercepts the data and prints the ASCII character to the simulator log. This hardware-software co-verification approach ensures that the output log accurately reflects the internal state of the processor pipeline.

4.1.3. Automated Instruction Set Compliance Results

Following the waveform analysis, the design was subjected to an automated regression test using the isa_4b.hex assembly test suite. This program exhaustively exercises the RV32I instruction set, performing a sequence of arithmetic, logic, branch, and memory operations. The simulation console output, generated by the Xcelium runner, serves as the pass/fail scoreboard.

As shown in the simulation log (Figure 4.2), the processor successfully executed all instruction groups:

Integer Arithmetic & Logic: Instructions such as add, sub, and, or passed, confirming the ALU integrity.

Memory Operations: The lw, sw, lh, sh, lb, sb tests passed, which validates the end-to-end data path through the TileLink adapters.

Control Flow: Branch and Jump instructions (beq, jal, jalr) operated correctly, ensuring the pipeline flush and PC update mechanisms are functional.

A significant milestone is the "PASS" result for the malgn (misalignment) test case. This automated check corroborates the visual findings from SimVision, confirming that the system can handle non-word-aligned accesses across the full memory range without data corruption.

```
        Design hierarchy summary:
                                Instances  Unique
                Modules:              193      52
                Verilog packages:       3       6
                Primitives:             5       1
                Registers:            788     522
                Scalar wires:        1179       -
                Expanded wires:       410      17
                Vectored wires:       590       -
                Always blocks:         38      28
                Initial blocks:        13      10
                Cont. assignments:    391     141
                Pseudo assignments:   827     464
                Compilation units:      1       1
                Simulation timescale:   1ps
        Writing initial simulation snapshot: worklib.mux_8:sv
Loading snapshot worklib.mux_8:sv ..................... Done
xmsim: *W,DSEM2009: This SystemVerilog design is simulated as per IEEE 1800-2009 SystemVerilog simulation semantics. Use -disable_sem2009 option for turning off SV 2009 simulation semantics
xcelium> source /opt/cadence/XCELIUM2009/tools/xcelium/files/xmsimrc
xcelium> run

PIPELINE - ISA tests

add......PASS
addi.....PASS
sub......PASS
and......PASS
andi.....PASS
or.......PASS
ori......PASS
xor......PASS
xori.....PASS
slt......PASS
slti.....PASS
sltu.....PASS
sltiu....PASS
sll......PASS
```

Figure 11. Xcelium Simulation Log

```
sll......PASS
slli.....PASS
srl......PASS
srli.....PASS
sra......PASS
srai.....PASS
lw.......PASS
lh.......PASS
lhu......PASS
lb.......PASS
lbu......PASS
sw.......PASS
sh.......PASS
sb.......PASS
auipc....PASS
lui......PASS
beq......PASS
bne......PASS
blt......PASS
bltu.....PASS
bge......PASS
bgeu.....PASS
jal......PASS
jalr.....PASS
malgn....PASS
iosw.....PASS
```

Figure 12. Xcelium Simulation Log (cont.)

4.1.4. Simulation Performance Metrics

The simulation log also provided quantitative performance metrics for the implemented pipeline. Over the course of the test program, the processor executed a total of 4,825 instructions in 7,830 clock cycles, resulting in an Instruction Per Cycle (IPC) ratio of **0.62**. The report indicates a Branch Misprediction Rate of 100.00%, which is expected given the static "Always Not Taken" prediction strategy combined with a test loop structure that is predominantly "Taken".

```
malgn....PASS
iosw.....PASS

================== Result ==================
Total Clock Cycles Executed = 7830
Total Instructions Executed = 4825
Total Branch Instructions   = 1450
Total Branch Mispredictions = 1450

------------------------------------------------
Instruction Per Cycle (IPC) = 0.62
Branch Misprediction Rate   = 100.00 %

END of ISA tests
```

Figure 13. Xcelium Simulation Performance Metrics

4.2. Synthesis and Resource Utilization

Following functional verification, the design was synthesized using Intel Quartus Prime (Standard Edition) to evaluate its physical implementation feasibility on the Intel Cyclone V FPGA (Device: 5CSXFC6D6F31C6). The synthesis process converts

the Register Transfer Level (RTL) description into a gate-level netlist mapped to the FPGA's logic resources.

4.2.1. Logic Utilization

The synthesis report indicates that the complete system—comprising the processor core, TileLink adapters, and memory interface logic—utilizes **1,990 Adaptive Logic Modules (ALMs)**, which represents approximately **5%** of the device's total logic capacity. Additionally, the design employs **910 dedicated logic registers** for pipeline staging and state machine control. These figures demonstrate that the integration of the TileLink-UL protocol introduces a manageable hardware overhead, leaving significant headroom for future architectural expansions such as floating-point units or hardware accelerators.

A critical aspect of the SoC implementation is the efficient use of on-chip memory. The synthesis results confirm the utilization of **526,336 block memory bits**, accounting for **9%** of the total available Block RAM (BRAM). This figure validates that the toolchain correctly inferred the 64 KB Data Memory and the Instruction Memory as low-latency dedicated memory blocks rather than distributed logic, ensuring optimal access times and reduced logic congestion.

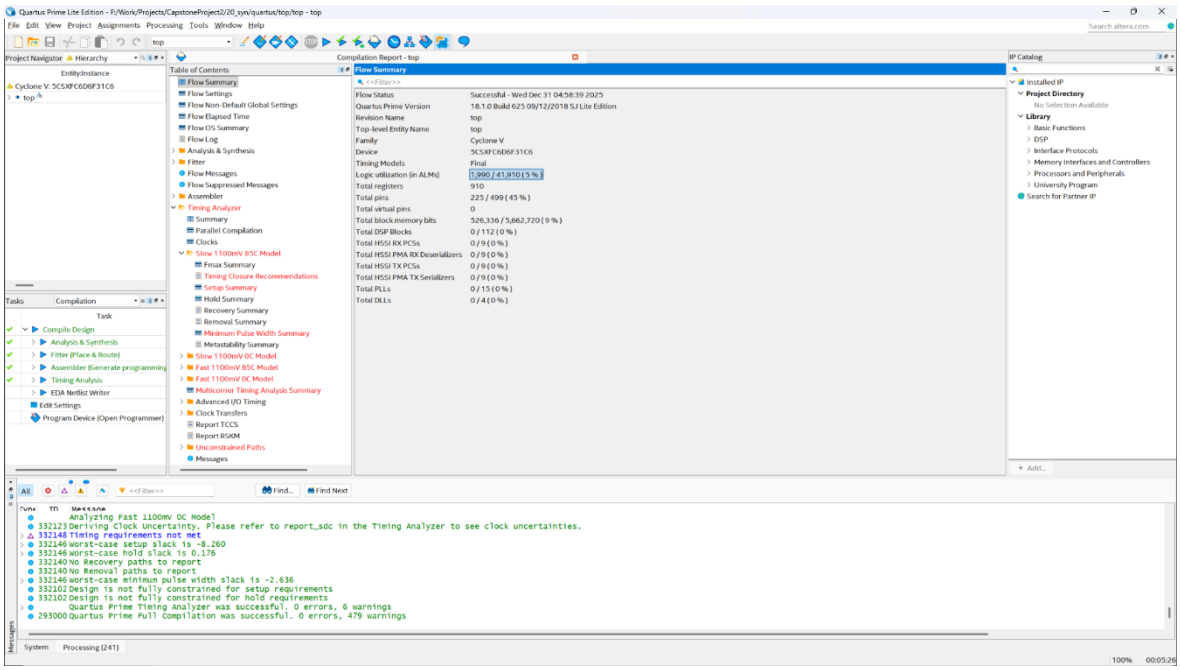


Figure 14. Logic Resource Utilization report indicating 5% ALM usage on the Cyclone V device

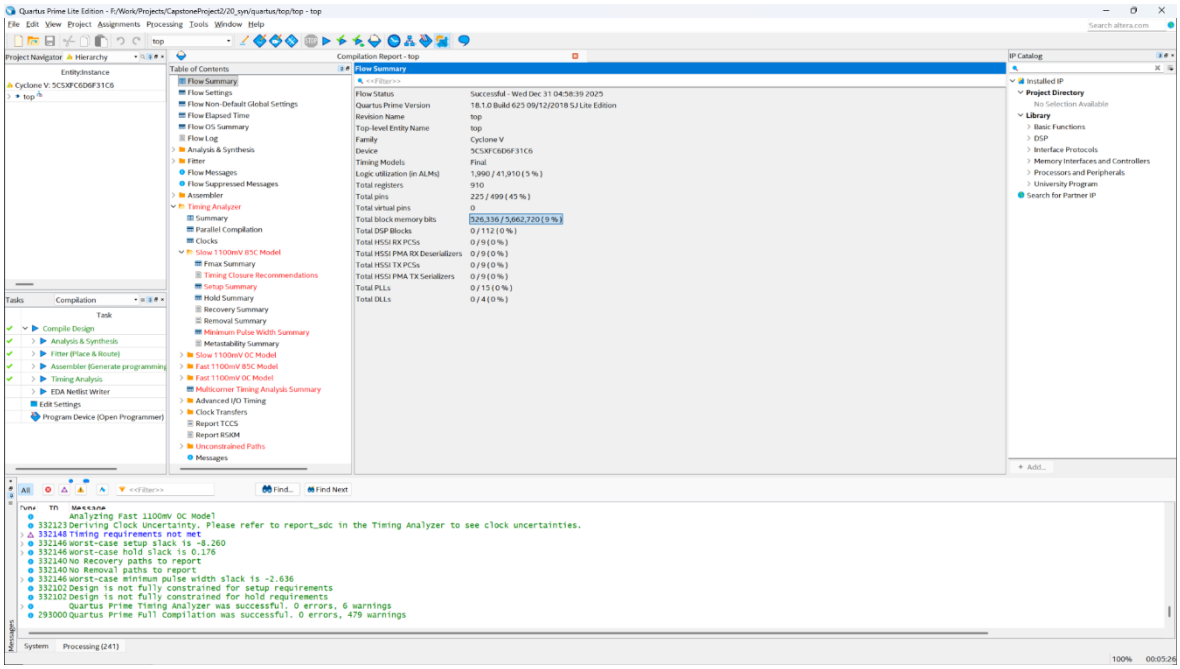


Figure 15. Memory Statistics confirming the inference of On-Chip Block RAM for Instruction and Data memories

4.2.2. I/O Connectivity

Regarding external connectivity, the design occupies **225 I/O pins**, utilizing **45%** of the available pin count. This extensive pin usage is attributed to the comprehensive mapping of the development board's peripherals. The processor's Memory-Mapped I/O (MMIO) controller is fully bonded to the 7-segment displays, LEDs, LCD interface, and slide switches, demonstrating a fully integrated embedded system capable of complex user interaction.

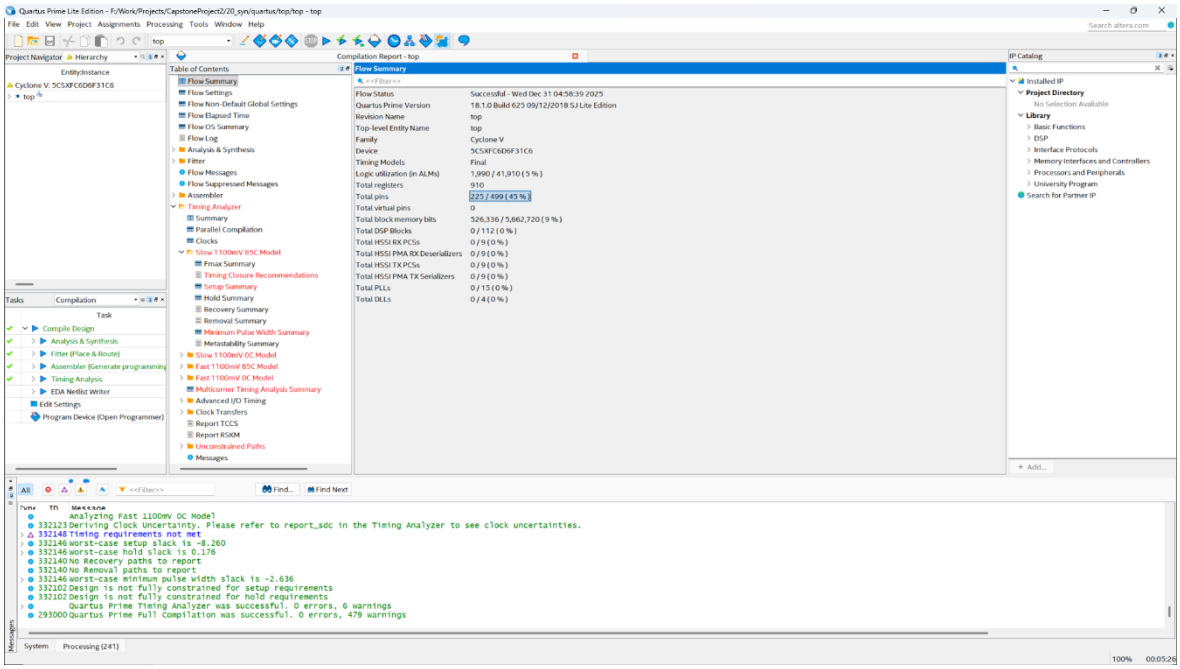


Figure 16. Quartus Pin Planner report illustrating the 45% I/O utilization for peripheral mapping

4.2.3. Timing Analysis

Static Timing Analysis (STA) was performed to determine the maximum operating frequency (F_{max}). The design achieved an F_{max} of **60.16 MHz** at the slow 85°C corner. This performance exceeds the target system clock of 50 MHz, providing a positive timing slack of roughly 3.4 ns. This safety margin ensures reliable operation even under varying thermal conditions or potential signal integrity fluctuations on the physical board.

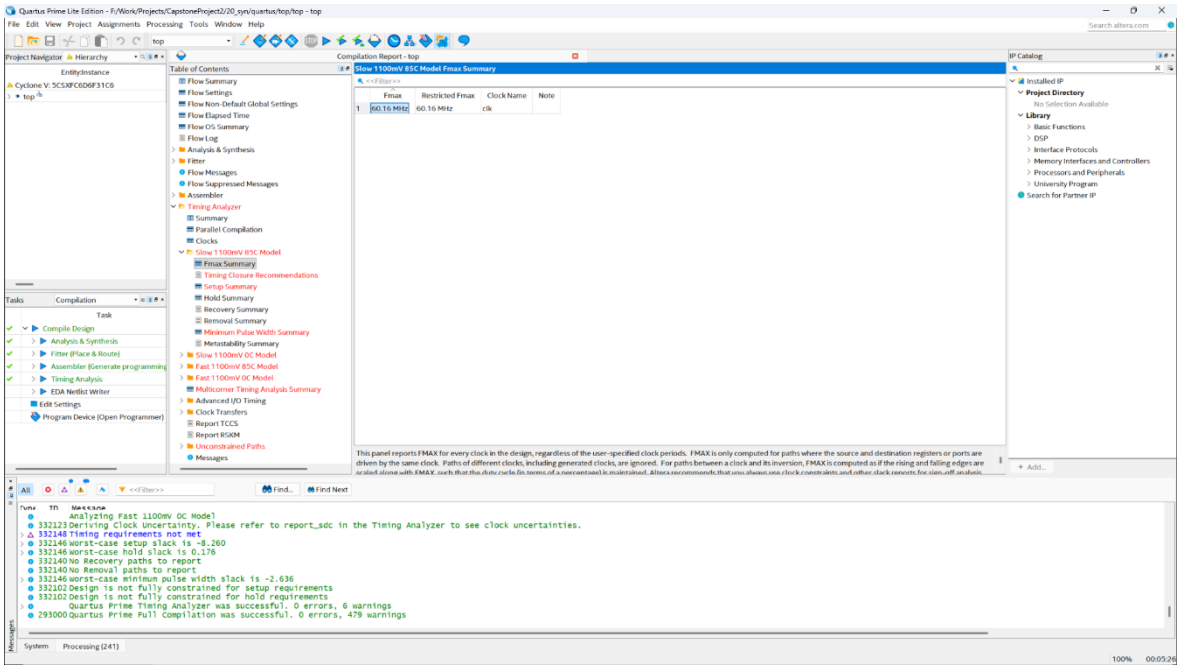


Figure 17. Fmax Summary report from TimeQuest Timing Analyzer confirming a maximum operating frequency of 60.16 MHz

4.2.4. Netlist Analysis

The RTL Viewer generated by Quartus visually confirms the structural hierarchy. The tlul_mgr block is clearly visible interposed between the processor and the memory blocks, verifying that the physical synthesis matches the intended logical architecture.

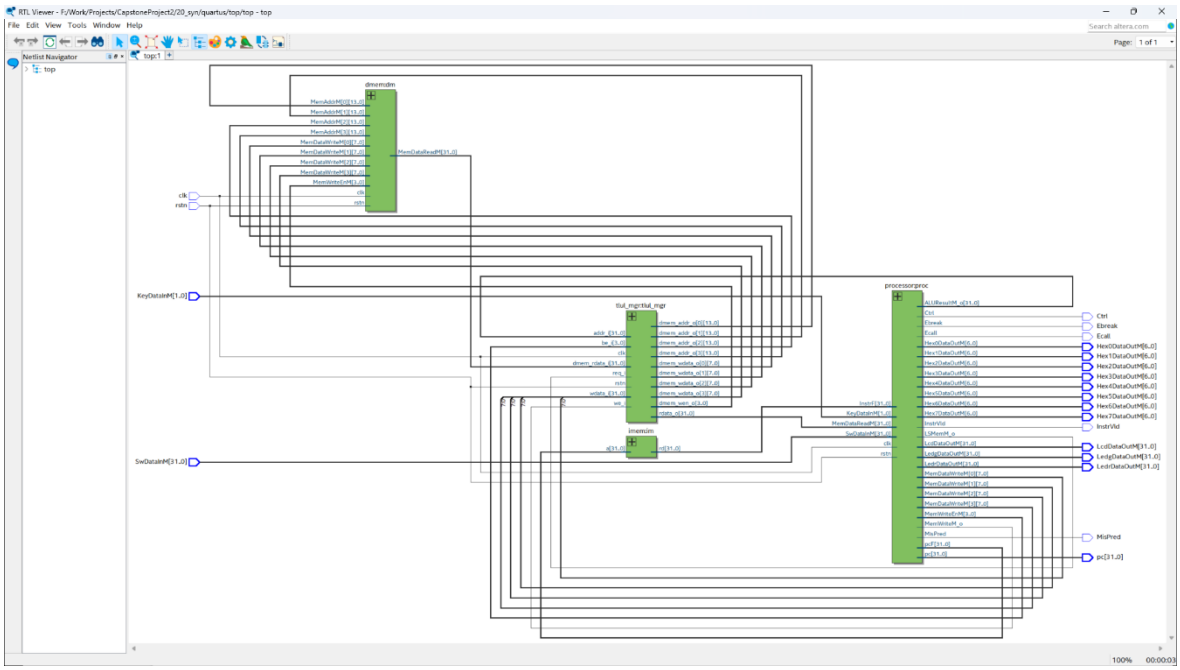


Figure 18. Gate-level netlist generated by RTL Viewer showing the interconnection between the RISC-V Core and the TileLink Manager

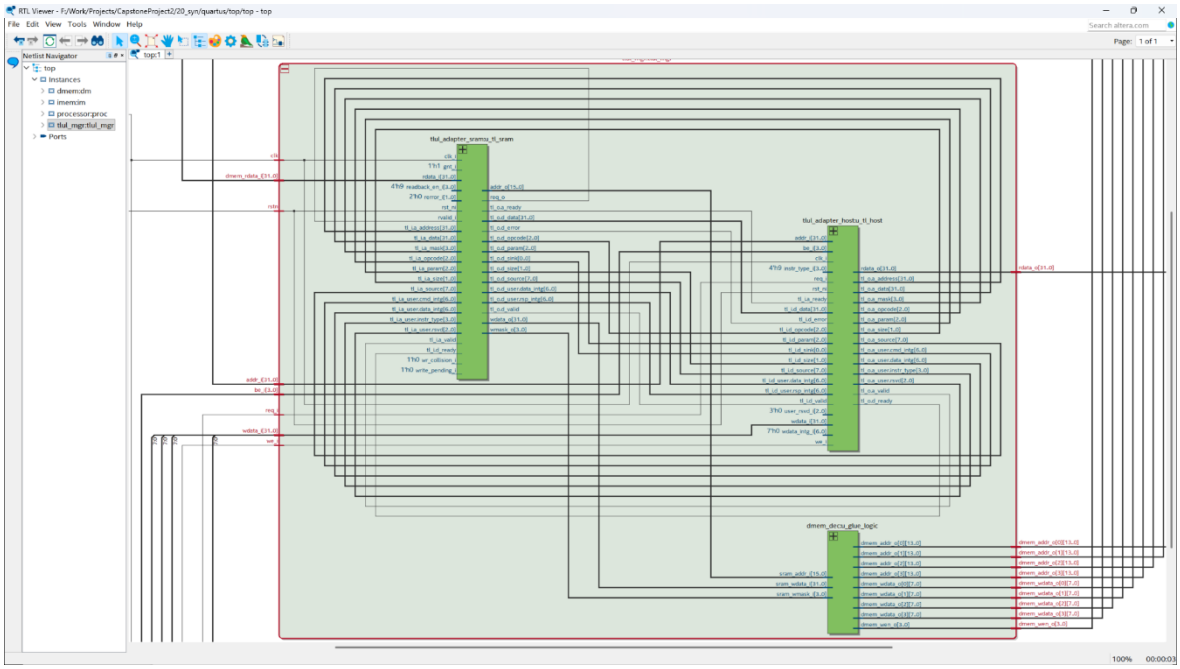


Figure 19. Internal RTL structure of the tlul_mgr module showing the interconnect between Host Adapter, Device Adapter, and Address Decoupling logic

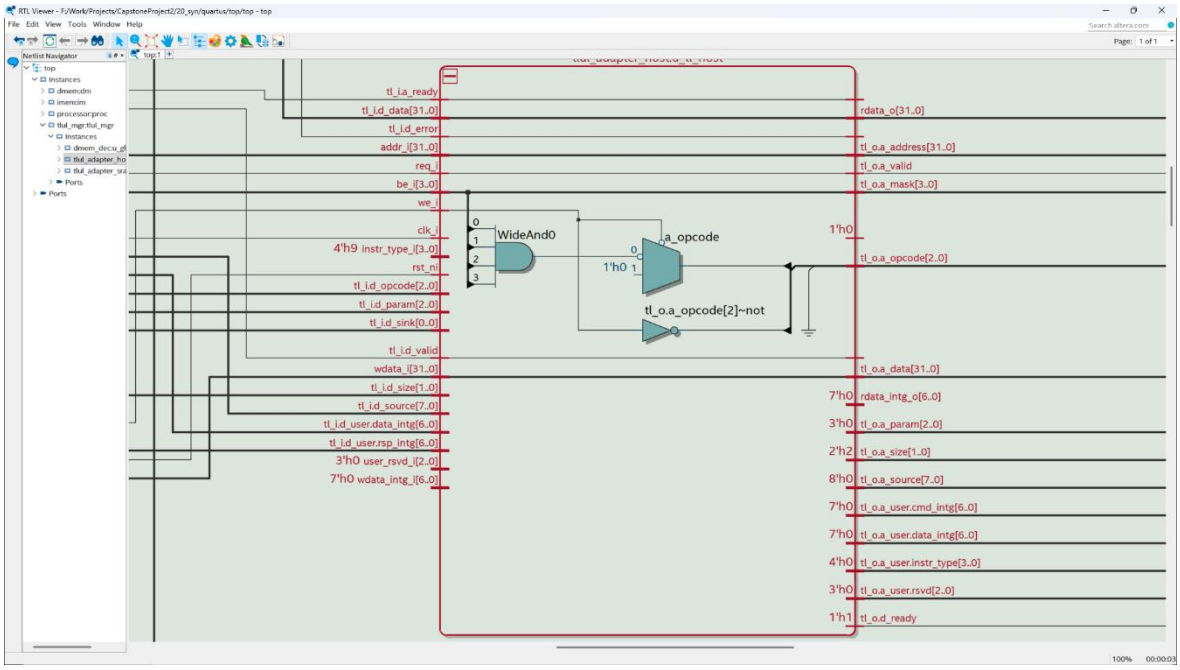


Figure 20. Synthesized Netlist of the tlul_adapter_host module illustrating the request generation logic and TileLink Channel A mapping

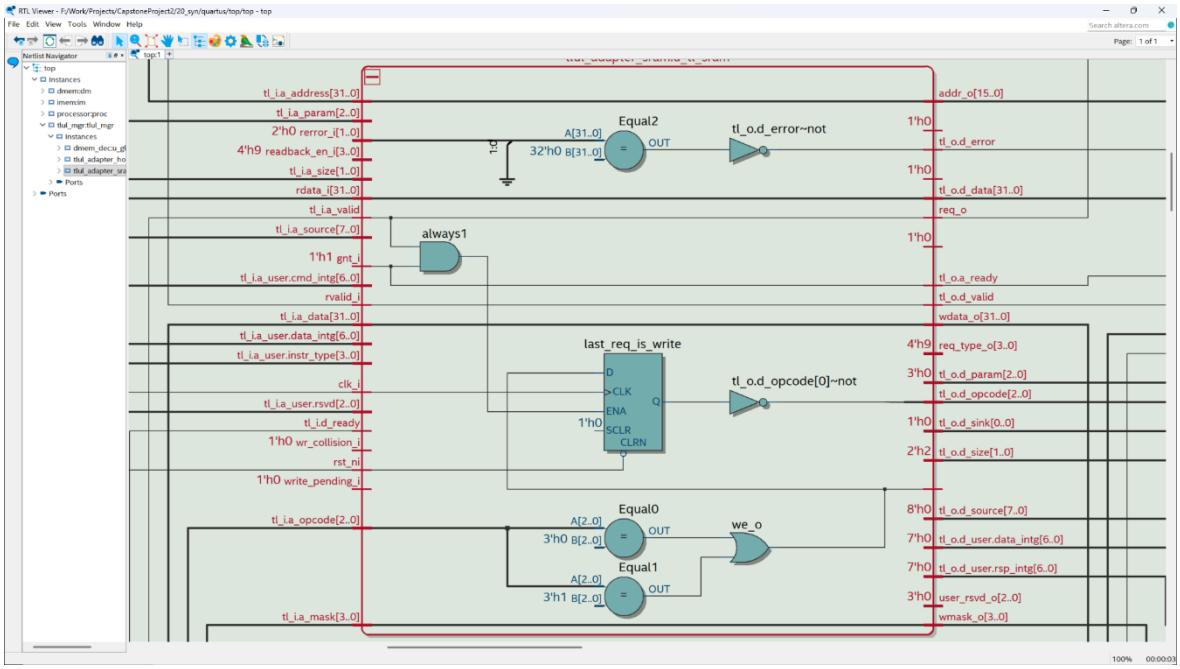


Figure 21. Gate-level Netlist of the tlul_adapter_sram module showing the transaction decoding and SRAM interface control signals

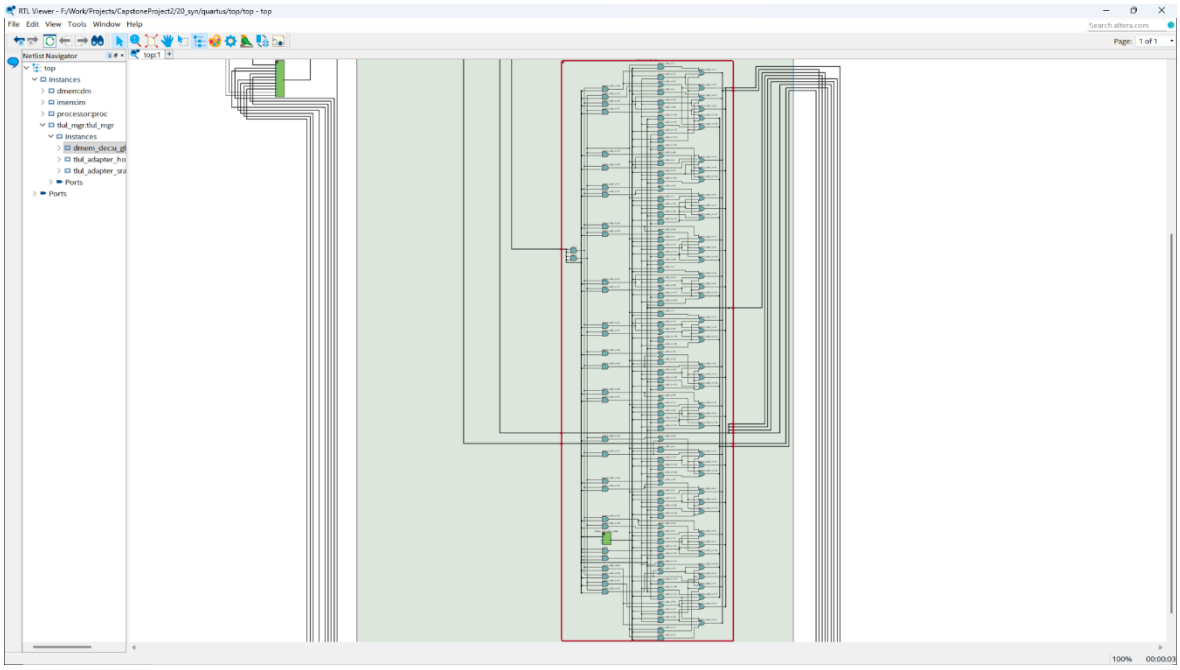


Figure 22. Detailed Netlist View of the dmem_dec (Glue Logic) module highlighting the address swizzling arithmetic and bank selection multiplexers

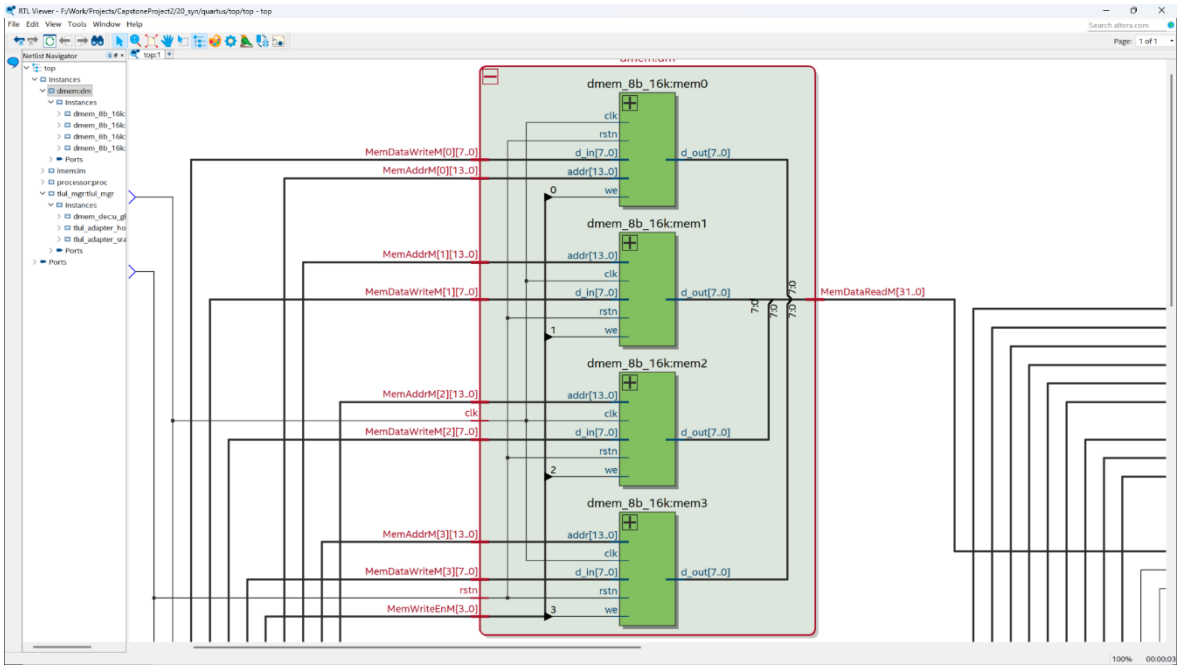


Figure 23. Synthesized Netlist of the Data Memory (dmem) module showing the interleaved organization of four 8-bit memory banks

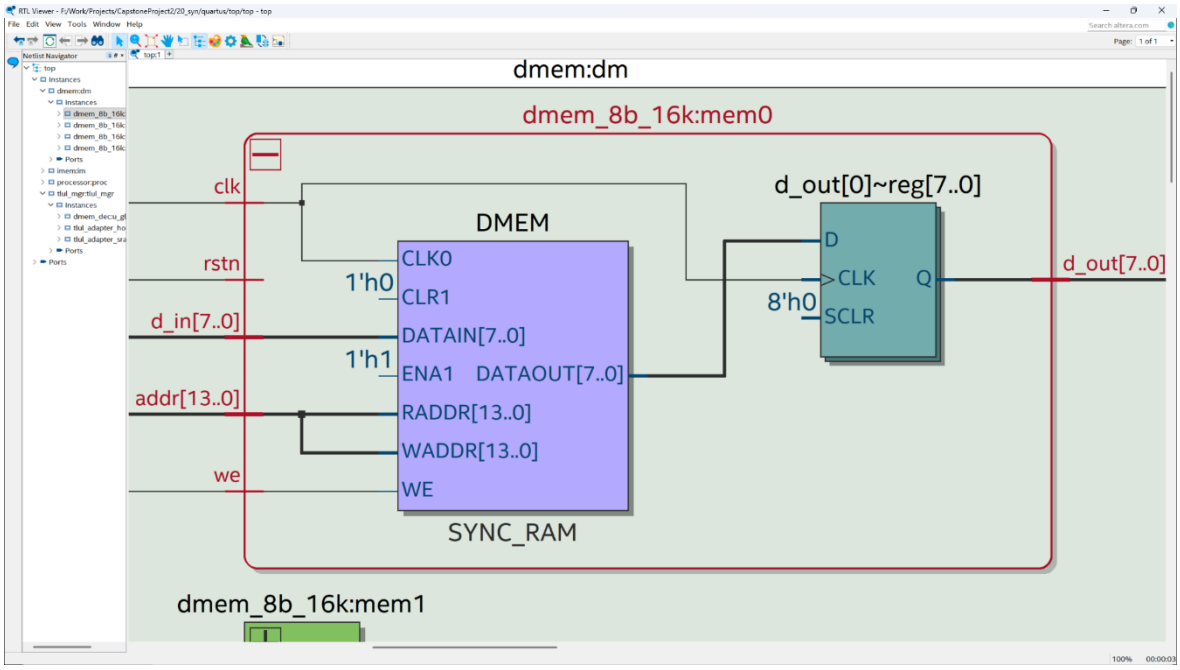


Figure 24. Internal view of a single dmem_8b_16k bank showing the primitive Altera/Intel synchronization RAM blocks

4.3. FPGA Hardware Implementation

To validate the design in a real-world environment, the synthesized bitstream was downloaded onto the DE10-Standard FPGA development board. A "Stopwatch" demonstration application was developed to test the system's Input/Output capabilities.

4.3.1. Setup and Methodology

The setup involved mapping the processor's Memory-Mapped I/O (MMIO) ports to the board's physical peripherals. The 7-segment displays and LCD were used as output devices, while the push buttons served as input interrupts/polling sources. The TileLink adapters were responsible for routing these I/O requests to the appropriate peripheral controllers.

4.3.2. Experimental Results

The stopwatch application functioned correctly on the hardware. The LCD displayed the time counter incrementing in real-time, and the buttons successfully triggered start, stop, and reset functions. The stability of the display updates confirms that the bus protocol correctly handles transaction timing and that the processor is successfully executing instructions at the 50 MHz clock speed without data hazards or bus contention.

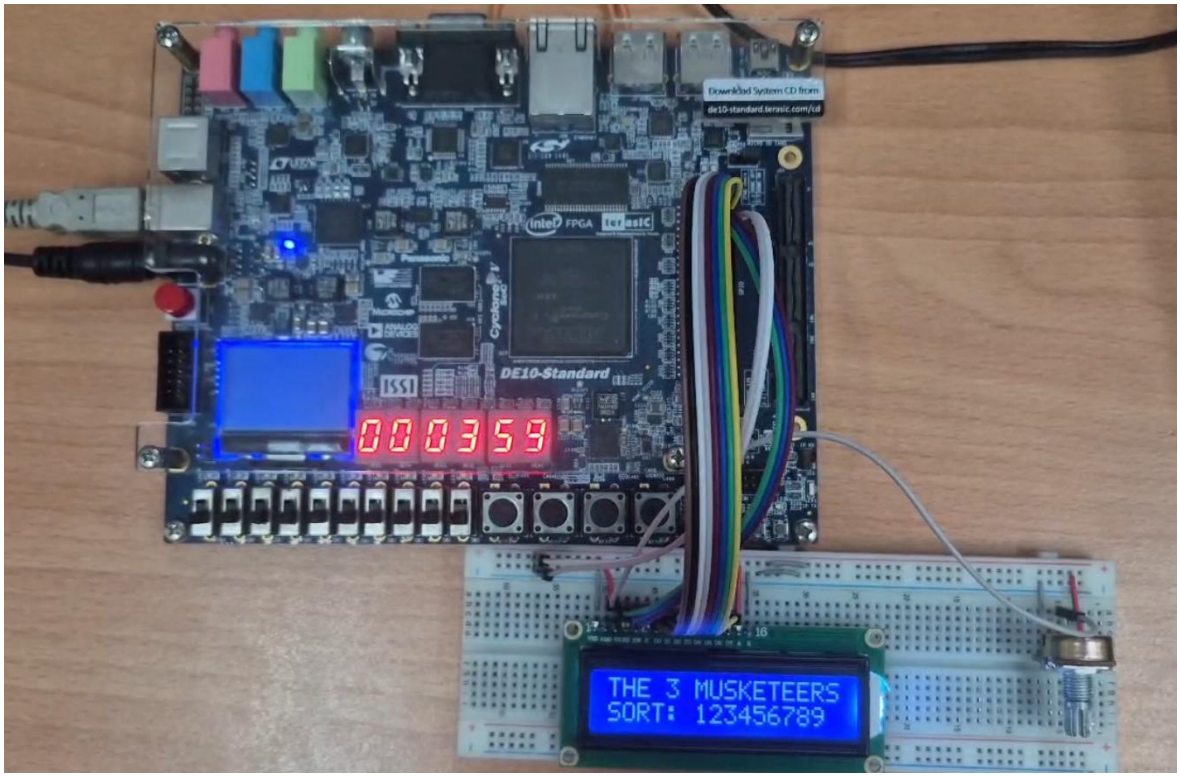


Figure 25. Experimental hardware setup utilizing the Intel Cyclone V FPGA on the DE10-Standard development board

5. PROJECT REPOSITORY

5.1. Overview of Repository Organization

The complete source code for this project, including the SystemVerilog RTL design, verification environment, and physical implementation scripts, is hosted and version-controlled on GitHub at the following repository:

Repository URL: https://github.com/ndNgoc1310/CapstoneProject2

To ensure design modularity, reproducibility, and ease of collaboration, the repository is organized into a strict hierarchical directory structure. This structure separates the design source code (RTL) from the verification environments and physical implementation files. The root directory is organized as follows:

00_rtl: This is the core directory containing the synthesizable SystemVerilog source code. It is further subdivided into functional units such as core (processor pipeline), memory (LSU and memory arrays), and tilelink (bus adapters and protocol managers). This separation ensures that the design logic remains independent of any specific EDA tool.

10_sim: This directory contains the verification environment. It houses the testbench files, assembly test programs (such as isa_4b.hex), and scripts required to run simulations on Cadence Xcelium.

20_syn: This directory is dedicated to physical implementation. It contains the Intel Quartus Prime project files (.qpf, .qsf) and the synthesis output reports. This isolation prevents synthesis artifacts from cluttering the source directories.

91_scripts: This folder contains utility scripts (Batch/Shell) used for automating build processes and synchronizing source files between different environments.

5.2. Version Control Methodology

The integrity of the design throughout the development lifecycle was maintained using Git version control. The project utilized a "Feature Branch" workflow to manage the integration of the TileLink protocol without disrupting the stability of the baseline processor.

Specifically, the baseline 5-stage RISC-V processor was maintained on the main branch. The development of the bus interconnect was isolated in a dedicated branch named feature/tilelink-integration. This approach allowed for the incremental development and testing of the tlul_adapter modules. Only after the simulation confirmed that the adapters correctly handled misaligned accesses and protocol handshakes was the code merged. This methodology provides a clear history of changes and facilitates the rollback of code in case of regression errors.

5.3. Version Control Methodology

A challenge encountered during the project was the discrepancy between the hierarchical source structure preferred for development and the flat file structure often required by synthesis tools or remote simulation servers. To address this, the project employs automated synchronization scripts (e.g., sync_src_to_quartus.bat).

These scripts perform two critical functions. First, they aggregate all SystemVerilog source files (.sv) and headers (.svh) from the nested subdirectories of 00_rtl. Second, they flatten and copy these files into the target build directories (20_syn or 10_sim). This automation ensures that the simulation and synthesis tools always operate on the most up-to-date version of the code, eliminating human error associated with manual file copying and dependency management.

6. CONCLUSION AND FUTURE VISION

6.1. Version Control Methodology

This project has successfully achieved its primary objective of decoupling the memory subsystem from the RISC-V processor core through the integration of the TileLink Uncached Lightweight (TL-UL) bus protocol. By replacing the legacy direct-connection architecture with a modular, standard-compliant interconnect, the design has transitioned from a standalone educational processor to a scalable System-on-Chip (SoC) foundation.

The most significant technical contribution of this work lies in the robust implementation of the **TileLink Interconnect Manager** and the associated **Address Decoupling Logic**. The design successfully resolved the impedance mismatch between the flat, word-aligned addressing scheme of the bus and the interleaved, byte-banked architecture of the physical memory. The simulation results, verified through both visual waveform analysis in SimVision and automated self-checking assembly tests, confirmed that the system correctly handles all memory access scenarios, including complex misaligned store operations, without data corruption or pipeline hazards.

Furthermore, the physical feasibility of the design was validated through synthesis and implementation on the Intel Cyclone V FPGA. The resource utilization report indicated that the bus integration introduces only a marginal overhead (approximately 5% logic utilization), while maintaining a high operating frequency ($F_{max} > 60$ MHz). The successful deployment of the stopwatch application on the DE10-Standard kit serves as empirical evidence that the processor, adapters, and peripheral interfaces operate synchronously in a real-world hardware environment.

In summary, this project has not only enhanced the modularity and extensibility of the RISC-V core but has also established a verified hardware framework that adheres to industry standards, ready for complex peripheral integration.

6.2. Future Vision

While the current implementation establishes a functional point-to-point bus connection, the full potential of the TileLink protocol lies in its network capabilities. To evolve this design into a commercial-grade microcontroller, several future development phases are proposed:

Implementation of a Bus Crossbar (1:N Interconnect): The immediate next step is to replace the single-slave topology with a 1:N Bus Crossbar. This will allow the processor (Master) to communicate with multiple distinct slaves simultaneously based on the address map. This expansion is crucial for integrating essential communication peripherals such as UART, SPI, and I2C, enabling the processor to interact with external sensors and devices.

Integration of Instruction Cache (I-Cache): Currently, the Instruction Fetch stage accesses memory directly, which may become a bottleneck as bus traffic increases. Future work should involve designing a TileLink Host Adapter for the Instruction Fetch unit, coupled with a Level-1 Instruction Cache. This would significantly improve IPC (Instructions Per Cycle) by reducing bus contention between instruction fetches and data load/store operations.

DMA Controller Integration: To further offload data movement tasks from the CPU, a Direct Memory Access (DMA) controller should be integrated as a second Bus Master. This would require upgrading the Crossbar to an M:N topology, allowing the DMA to transfer data between peripherals and memory while the CPU executes instructions in parallel, thereby maximizing system throughput.

Advanced Protocol Features: The current design utilizes the basic request-response features of TL-UL. Future iterations could explore the **TileLink Uncached Heavyweight (TL-UH)** profile to support burst transactions. Burst capability would allow the processor to transfer multiple data words in a single request handshake, significantly optimizing bandwidth for cache-line refills or block memory transfers.

7. REFERENCES

[1] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Waltham, MA: Morgan Kaufmann, 2021.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2020.

[3] A. Waterman and K. Asanović, Eds., "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," Document Version 20191213, RISC-V International, Dec. 2019. [Online]. Available: https://riscv.org/technical/specifications/

[4] SiFive, Inc., "SiFive TileLink Specification," Version 1.8.0, Aug. 2019. [Online]. Available: https://sifive.cdn.prismic.io/sifive/57f93ecf-2c42-46f7-9818-bcdd7d3ed29c_tilelink-spec-1.8.0.pdf

[5] lowRISC Contributors, "OpenTitan Technical Documentation: TileLink Uncached Lightweight (TL-UL)," OpenTitan Project, 2023. [Online]. Available: https://opentitan.org/book/hw/ip/tlul/index.html

[6] Intel Corporation, "Cyclone V Device Handbook, Volume 1: Device Interfaces and Integration," San Jose, CA, 2016.

[7] IEEE Standard Association, "IEEE Standard for SystemVerilog--Unified Hardware Design, Specification, and Verification Languages," IEEE Std 1800-2017, Feb. 2018.